Expert Systems With Applications 330 (2026) 133020



Contents lists available at ScienceDirect

Expert Systems With Applications

journal homepage: www.elsevier.com/locate/eswa





Dual-role LLMs for evolutionary neural architecture search: Evolutionary generation and comparative prediction

Yandan Xu [a,b], Weilin Fang [b,*], Yu Xue [b]

[a] Huzhou Vocational and Technical College, 3999 Chuangye Road, Huzhou, 313000, China
[b] School of Software, Nanjing University of Information Science and Technology, Nanjing, 210000, China

ARTICLE INFO

Keywords:
Surrogate model
Evolutionary algorithm
Large language model
Neural architecture search

ABSTRACT

Evolutionary neural architecture search (ENAS) automates network design but suffers from high computational costs. Existing surrogate-assisted approaches struggle to balance cost and accuracy, often failing in data-scarce scenarios due to their heavy reliance on extensive ground-truth data. Additionally, traditional evolutionary operators, driven by numerical formulas or stochastic perturbations, often ignore topological semantics, leading to inefficient and undirected search. To address these challenges, Dual-Role LLMs for evolutionary neural architecture search (DR-LLM-ENAS) is proposed. First, a semantic evolutionary update mechanism is introduced, which substitutes the numerical state update formulas of traditional particle swarm optimization with the code reasoning capabilities of LLMs. This mechanism utilizes natural language to guide the directional mutation and generation of architectures. Second, an in-context learning performance predictor is designed. By dynamically retrieving the historical evaluation data of the population to construct a context, this predictor leverages in-context learning to achieve efficient, gradient-free architecture performance prediction. Extensive experiments on the NAS-Bench-201 search space demonstrate that DR-LLM-ENAS achieves performance superior to current state-of-the-art methods. Additionally, results on the DARTS and NAS-Bench-NLP search spaces further validate the effectiveness and generalization capability of our approach. Specifically, the method achieves approximately 3% relative improvement over state-of-the-art methods on the DARTS search space. The code is publicly available at: https://github.com/baigeixiaowang/DR-LLM-NAS.

1. Introduction

Deep neural networks have achieved remarkable success in numerous frontier domains, such as medical imaging analysis (Okolo et al., 2025) and computer vision (Yu et al., 2024). However, as model parameter counts and topological complexity surge, the traditional architecture design paradigm, which relies heavily on expert prior knowledge and manual trial-and-error, struggles to meet the increasingly diverse demands of various tasks. As an automated alternative, neural architecture search (NAS) is dedicated to discovering optimal architectures for specific task datasets within vast search spaces, emerging as a critical technique for advancing deep learning.

Among various NAS strategies, evolutionary algorithms (EAs) have garnered widespread attention due to their gradient-free nature and exceptional global search capabilities. However, existing evolutionary neural architecture search (ENAS) methods (Pham et al., 2018) predominantly employ stochastic crossover or mutation operators to generate candidate architectures. Such search strategies, based on random perturbations, often overlook the intrinsic semantics underlying the combinations of neural operators, yielding numerous invalid or redundant candidates. The prohibitive cost of training and testing each candidate severely constrains search efficiency. For instance, the evolution-based NAS developed by Real et al. (2017) required 450 GPUs running for 4 days. To alleviate this computational bottleneck, researchers have introduced surrogate models to predict architecture performance, leading to a surge of related works in recent years. For example, Xiang et al. (2024) proposed a hybrid architecture predictor combining graph neural networks and transformers. By leveraging GNNs to extract underlying local features and transformers to capture high-level global dependencies, they achieved precise regression of architecture accuracy without full training. Hwang et al. (2024) focused on the encoder design of surrogate models, introducing a flow-aware graph transformer named FlowerFormer. By simulating the actual data flow within neural architectures during the encoding phase, it provides more discriminative embedding representations for downstream performance predictors. Addressing prediction-based NAS, Akhauri and Abdelfattah (2024)

* Corresponding author.
E-mail addresses: yandan@huvtc.edu.cn (Y. Xu), fangwl@nuist.edu.cn (W. Fang), xueyu@nuist.edu.cn (Y. Xue).

https://doi.org/10.1016/j.eswa.2026.133020
Received 16 January 2026; Received in revised form 3 May 2026; Accepted 25 May 2026
Available online 4 June 2026
0957-4174/© 2026 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

proposed a unified encoding scheme named FLAN, utilizing a flow attention mechanism to simultaneously capture operational and topological information, effectively enhancing the ranking correlation of surrogate models across different search spaces. Furthermore, targeting the specific domain of diffusion models, Li et al. (2023) designed a training-free evolutionary search strategy in AutoDiffusion. Instead of relying on expensive FID score calculations, it directly optimizes the time steps and model architecture of the diffusion process, employing fast search metrics as performance surrogates to guide the evolution. Nevertheless, current surrogate-assisted ENAS methods still face severe challenges: On one hand, the prediction accuracy of surrogate models relies heavily on the quantity and quality of training samples, often falling into a "cold-start" problem during the initial search phase due to a lack of sufficient evaluation data, which leads to ineffective guidance; on the other hand, the design, training, and hyperparameter tuning of high-performance surrogate models themselves are deeply dependent on expert experience, which contradicts the original intention of NAS to liberate experts from tedious architecture design.

In recent years, the rise of large language models (LLMs) has provided a novel perspective for addressing these challenges. LLMs demonstrate powerful logical reasoning and code comprehension capabilities, achieving significant progress in algorithm design (Yu et al., 2025), combinatorial optimization (Yang et al., 2024), and integration with evolutionary computation (Liu et al., 2024). Based on pre-training with massive code data, LLMs exhibit an implicit capacity to capture the functional semantics of neural operators and their intricate topological compositions. This study aims to explore how to effectively transfer this structured prior knowledge to ENAS tasks, thereby achieving efficient, low-cost architecture search. The DR-LLM-ENAS framework is proposed in this study, with its core idea being to employ the LLM in a dual role: as an "evolutionary generator" and an "architecture comparative predictor." First, a semantic-aware evolutionary update mechanism is constructed. Unlike traditional physical formula-driven approaches, the LLM generates new architectures with higher potential using natural language reasoning based on the population's historical state. This process generates detailed reasoning logs, providing interpretability to the black-box search process. Second, the LLM is utilized as an architecture comparative predictor to address the issue of evaluation efficiency. Addressing the characteristic that LLMs have low sensitivity to absolute numerical values but excel in logical comparison, we propose a "comparative prediction strategy." This strategy decomposes complex absolute performance regression into a process of first determining relative superiority and then mapping numerical values based on these relative relationships.

DR-LLM-ENAS holds significant practical value for industrial deployment. Its training-free nature renders it particularly suitable for resource-constrained scenarios, such as automated model design on edge devices or mobile platforms where labeled architectural data is scarce and computational budgets are strictly limited. Furthermore, the natural language reasoning logs generated during the semantic evolutionary process provide high interpretability, which is crucial for safety-critical domains like medical diagnosis or autonomous driving that demand transparent decision-making and auditable model development trace. Finally, the inherent cross-space transferability offers a flexible framework for rapidly adapting to emerging tasks beyond computer vision—such as natural language processing architectures or time-series forecasting network designs—potentially accelerating the industrial adoption of automated machine learning in diverse fields.

The remainder of this paper is organized as follows. Section 2 reviews related work on PSO-based NAS and LLM-assisted optimization. Section 3 details the proposed DR-LLM-ENAS framework: Section 3.1 presents the overall dual-role mechanism, Section 3.2 describes LLM-guided semantic architecture generation, Section 3.3 introduces the LLM as comparative predictor, and Section 3.4 illustrates the differences between semantic evolution and traditional numerical mutations through concrete cases. Section 4 reports experimental results, including evaluations on three search spaces (NAS-Bench-201, DARTS, and NAS-Bench-NLP in Sections 4.1–4.4), comprehensive ablation studies (Section 4.5), and robustness analysis (Section 4.6). Section 5 concludes the paper with a discussion of limitations and future research directions.

The main contributions of this study are as follows:

1. A semantic evolution mechanism is proposed. By substituting traditional numerical operators in particle swarm optimization (PSO) with LLM-based semantic mutations, algorithmic convergence is significantly accelerated. Additionally, an anti-collapse strategy is introduced to maintain population diversity and prevent premature convergence to local optima.
2. A training-free, context-aware comparative predictor is designed. Unlike conventional surrogate models requiring extensive training, chain-of-thought (CoT) reasoning is leveraged to harness LLMs for architecture evaluation. This approach decomposes evaluation into relative superiority determination followed by precise performance regression.
3. Transparency and interpretability are enabled in the search process. By generating natural language reasoning logs for each evolutionary step, explicit insights are provided into the ENAS decision-making logic. Furthermore, these logs are utilized as prior knowledge to guide the updates of subsequent particles.

2. Related work

This section introduces the research foundations related to the methodology presented in this paper: Section 2.1 outlines the algorithmic process of PSO-based NAS and its formulation. Section 2.2 summarizes the integration of LLMs into evolutionary computation, specifically detailing their applications as evolutionary optimizers and surrogate models.

2.1. PSO-based NAS algorithms

NAS aims to automate the discovery of optimal neural network architectures for specific tasks, a process that is typically formulated as a bilevel optimization problem. Let A denote the search space, where each element $\alpha \in A$ represents a specific network architecture. The primary objective of NAS is to identify an optimal architecture α^* that minimizes the loss function on the validation set, denoted as L_{val}. This optimization is constrained by the requirement that the network weights must be optimized on the training set. The mathematical formulation is expressed as follows:

$$\begin{aligned} &\min_{\alpha \in A} \quad L_{val}\left(w^*(\alpha), \alpha\right) \\ &\text{s.t.} \quad w^*(\alpha) = \arg\min_{w} L_{train}(w, \alpha) \end{aligned} \tag{1}$$

where L_{train} represents the loss function on the training set, and w denotes the weight parameters associated with architecture α. Consequently, obtaining the optimal weights $w^*(\alpha)$ for each candidate architecture requires a complete training process. Given that the search space A is typically discrete and vast, and the evaluation of each α incurs significant computational costs, solving this problem via brute-force search or conventional gradient-based methods is often intractable.

To navigate this high-dimensional search space efficiently, PSO (Kennedy & Eberhart, 1995) is widely adopted due to its collaborative population-based search mechanism. PSO mimics the social behavior of bird flocking, leveraging collaboration and information sharing among individuals to search for global optima. In this algorithm, each candidate solution is referred to as a "particle." Consider a swarm consisting of N particles in a D-dimensional search space. For the ith particle at iteration t, its position is represented by the vector $x_i^t = (x_{i,1}^t, x_{i,2}^t, \ldots, x_{i,D}^t)$, and its flight velocity is denoted by $v_i^t = (v_{i,1}^t, v_{i,2}^t, \ldots, v_{i,D}^t)$. The particles dynamically adjust their trajectories based on their personal historical

best position ($pbest_i$) and the global historical best position of the entire swarm ($gbest$). The update mechanisms for velocity and position are governed by the following equations:

$$v_{i,d}^{t+1} = \omega \cdot v_{i,d}^{t} + c_1 \cdot r_1 \cdot \left(pbest_{i,d}^{t} - x_{i,d}^{t} \right) + c_2 \cdot r_2 \cdot \left(gbest_{d}^{t} - x_{i,d}^{t} \right) \quad (2)$$

$$x_{i,d}^{t+1} = x_{i,d}^{t} + v_{i,d}^{t+1} \quad (3)$$

where ω is the inertia weight, which balances the trade-off between global exploration and local exploitation; c_1 and c_2 are the cognitive and social coefficients, respectively, representing the particle's reliance on its own experience versus the swarm's experience; r_1 and r_2 are random numbers uniformly distributed in $[0, 1]$, introducing stochasticity to the search process.

2.2. Large language models

2.2.1. LLM-assisted evolutionary algorithm

The integration of LLMs with evolutionary computation has recently catalyzed a paradigm shift from stochastic heuristics toward knowledge-driven optimization. Early explorations primarily leveraged the code generation capabilities of LLMs, substituting traditional crossover and mutation operators to produce syntactically correct architectural variants, or inferring high-quality solutions directly through in-context learning within iterative dialogues. Building upon these foundations, a research paradigm oriented toward LLMs as direct optimizers has gradually emerged: by designing iterative prompt interaction mechanisms, studies have utilized the inherent CoT reasoning capabilities of models to analyze population distributions and guide search trajectories, achieving training-free optimization without reliance on specific evolutionary operators (Liu et al., 2024); subsequently, prompting strategies that combine optimization task descriptions with historical high-performing trajectories have enabled models to identify latent patterns in the solution space and infer novel solutions (Yang et al., 2024). Recent advancements have further introduced reflective hyper-heuristic frameworks that integrate short-term memory mechanisms, allowing LLMs to autonomously generate heuristic algorithm codes while continuously producing high-performance algorithmic logic through explicit reflection on superior and inferior cases from historical iterations (Ye et al., 2024). Nevertheless, these methods generally constrain LLMs to single functional roles—either as passive generators of architectural variants or as standalone optimizers—failing to fully exploit the potential for synergistic integration of generative and evaluative capabilities, which motivates the Dual-Role framework proposed in this study.

2.2.2. LLMs as surrogate models

Traditional surrogate-assisted evolutionary algorithms rely on manually crafted neural networks to approximate fitness landscapes, requiring not only substantial ground-truth data for supervised training but also suffering from severe cold-start issues during initial search phases. LLMs, with their exceptional in-context learning capabilities and pre-trained domain knowledge, provide a novel pathway for establishing "training-free" performance prediction mechanisms, fundamentally eliminating dependencies on task-specific model training. In this direction, existing research has systematically evaluated the impact of prompt engineering strategies, validating the feasibility of utilizing contextual exemplars to guide LLMs in fitness ranking (Hao et al., 2024); to further enhance discriminative capacity, multimodal feature fusion strategies have been proposed, combining semantic embeddings derived from model source code with structural features from graph neural networks (Selvam et al., 2024); addressing cross-domain transferability challenges, recent frameworks have leveraged LLMs to mine universal architectural design principles from source tasks, applying these rules to target domains through CoT reasoning to enable efficient architecture search under few-shot conditions (Zhou et al., 2025); additionally, LLM-based surrogates have been extended to specialized domains such as graph neural architecture search, employing sophisticated prompt engineering to solve combinatorial optimization problems (Zhang et al., 2025). Despite these efforts, existing methods primarily focus on absolute numerical regression or direct feature-level fusion, overlooking the inherent advantages of LLMs in relative comparison and logical reasoning, which directly inspires our adoption of a comparative prediction strategy that decomposes performance evaluation into a two-stage process of "relative superiority determination followed by contextual score mapping", better aligning with the cognitive characteristics of LLMs.

3. Methodology

3.1. Overall framework

As illustrated in Fig. 1, the DR-LLM-ENAS framework adheres to the traditional PSO paradigm while innovatively decoupling the LLM into two distinct roles: a generator (Role 1) and a predictor (Role 2). These roles are integrated into the semantic space of the PSO. To overcome the limitations of traditional evolutionary algorithms in understanding the topological semantics of architectures, our method maps the inertial, cognitive, and social components of PSO into dynamic prompts. This guides the LLM to perform architectural deduction based on CoT reasoning. This mechanism not only replaces blind stochastic mutation with deep semantic reasoning but also establishes a closed-loop process from global exploration to precise selection through dual-role interaction. The detailed procedure of DR-LLM-ENAS is presented in Algorithm 1, with the main steps outlined below:

(1) Initialization (Lines 1–5 of Algorithm 1)

The algorithm begins by initializing the population P, the archive Arc, and the inference log H. Specifically, all particles are randomly sampled and evaluated on the validation set. Based on these initial metrics, the system establishes the personal historical best p_{best} for each particle and the global best g_{best}. These initial states not only define the starting point of the search but also provide the critical baseline context for the subsequent semantic evolution.

(2) Role 1: Semantic Generation with PSO Context (Lines 7–10 of Algorithm 1)

In each evolutionary iteration, the LLM is activated as Role 1 to execute the architectural generation task. The system first serializes the current topological structure of the particle, p_{best}, and g_{best} into natural language prompts, constructing a semantic field containing the evolutionary direction. Upon receiving this prompt, the LLM analyzes potential topological features and integrates historical experience with swarm intelligence to generate a new candidate architecture. This effectively transforms the update logic from numerical space into a reasoning process within the semantic space.

(3) Anti-Collapse and Constraint Check (Lines 11–13 of Algorithm 1)

To prevent model collapse caused by the LLM overfitting to high-scoring samples, a strict diversity constraint mechanism is introduced. When a newly generated architecture is detected as a duplicate of historical records in Arc or identical to the current g_{best}, the system triggers a retry mechanism with penalty instructions. If the retry fails to meet diversity requirements, the algorithm automatically reverts to a random mutation strategy to enforce an effective distribution of the population within the search space.

(4) Role 2: Comparative Prediction (Lines 14–16 of Algorithm 1)

Before conducting expensive true evaluations, the LLM switches to Role 2 to perform performance forecasting on the new architecture. Through the CoT reasoning mechanism, Role 2 analyzes the potential performance gain of the new architecture relative to its parent. If the LLM assigns a low score to the current architecture, the true evaluation is skipped. This process enriches the contextual environment by accumulating reasoning knowledge, significantly enhancing the interpretability and decision-making capability of the algorithm in subsequent iterations.

(5) Evaluation and PSO Update (Lines 17–28 of Algorithm 1)

In the final phase, architectures that pass the screening are evaluated on the validation set to obtain their true accuracy and are synchronously

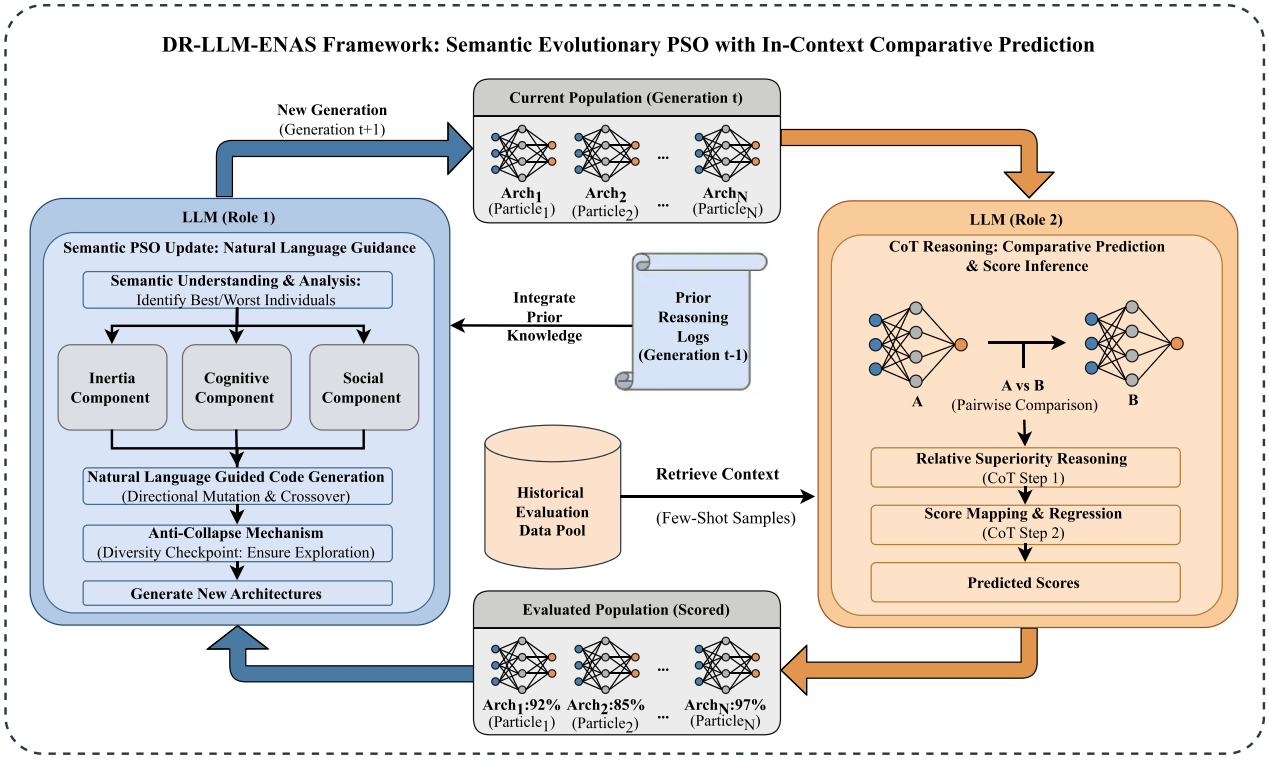


Fig. 1. An overview of the DR-LLM-ENAS framework.

updated to the archive Arc. Subsequently, the algorithm executes standard PSO update logic: comparing the accuracy of the new architecture with the particle's p_{best} and the current g_{best}. If the new architecture demonstrates superior performance, the corresponding historical best state is updated. This mechanism ensures that the population steadily and efficiently converges to the global optimum driven by semantic evolution.

3.2. LLM-guided architecture generation

This study proposes a semantic-driven evolutionary framework that reformulates the kinematic equations of PSO into an interpretable reasoning process governed by LLMs. In this framework, instead of relying on the linear weighting of physical vectors, we map the core evolutionary elements of PSO to concrete semantic operations. Specifically, the velocity of a particle is redefined as a semantic description of the evolutionary direction, which represents the concrete intent generated by the LLM to retain or modify specific operators, rather than an abstract numerical increment. The inertia component is transformed into an optimization consistency constraint, which employs prompt instructions to compel the model to maintain the effective structural design logic from the previous iteration, thereby suppressing irrational stochastic mutations. The cognitive component is implemented as a structural reflection and restoration mechanism, where the model deeply compares the current state with the personal best ($pbest$) to infer and recover effective components (such as specific skip connections) that were inadvertently discarded during exploration. The social component functions as structural motif transfer, in which the model actively extracts key topological features (such as deep convolutional stacking patterns) from the global best solution ($gbest$) and maps them onto the current architecture. This reformulation empowers the algorithm to exploit pre-trained linguistic priors for principled logical deduction across the architectural search space.

A structured prompt interaction mechanism is designed to translate these semantic operators into an executable algorithmic process, as illustrated in Fig. 2. This mechanism dynamically constructs inputs based on the evolutionary state, comprising four key components. First, role definition and space constraints explicitly enumerate the valid operation set to ensure the generated strings strictly adhere to the search space encoding rules. Second, state context injection provides the json descriptions of the current particle, $pbest$, and $gbest$, creating a multi-perspective comparative context. Third, the evolutionary strategy imposes a step-by-step reasoning chain that mirrors the PSO dynamics: it instructs the model to preserve current strengths (inertia), retrieve historical merits from $pbest$ (cognitive), mimic superior patterns from $gbest$ (social), and introduce necessary random perturbations (mutation). Finally, formatted output constraints compel the model to populate a standard json object containing both the design rationale and the new architecture, bridging the gap between semantic reasoning and numerical optimization.

The robustness of this prompt design stems from its adherence to the principle of static core instructions with dynamic context injection. The core instruction logic—including the semantic mappings of PSO components, mandatory CoT requirements, and strict JSON schema constraints—remains absolutely stable across all experiments, while only task-specific variables (e.g., {CURRENT_ARCH_STR}, {GBEST_ACC}) are dynamically populated via template placeholders. This separation ensures experimental reproducibility and behavioral consistency. To mitigate hallucinations and random errors, we enforce a deterministic reasoning trajectory through the step-by-step "inertia → cognitive → social" workflow (Fig. 2), compelling the LLM to articulate its deductive process before generating the final architecture. Furthermore, the generator operates at a moderate temperature = 0.7, balancing creative exploration with the stability required to maintain valid topological constraints.

Furthermore, the framework introduces a highly transparent knowledge accumulation mechanism by constructing explicit reasoning logs. During architecture generation, the LLM is mandated to output a decision rationale in natural language, detailing the reasons for specific operator replacements based on the provided context. These logs con-

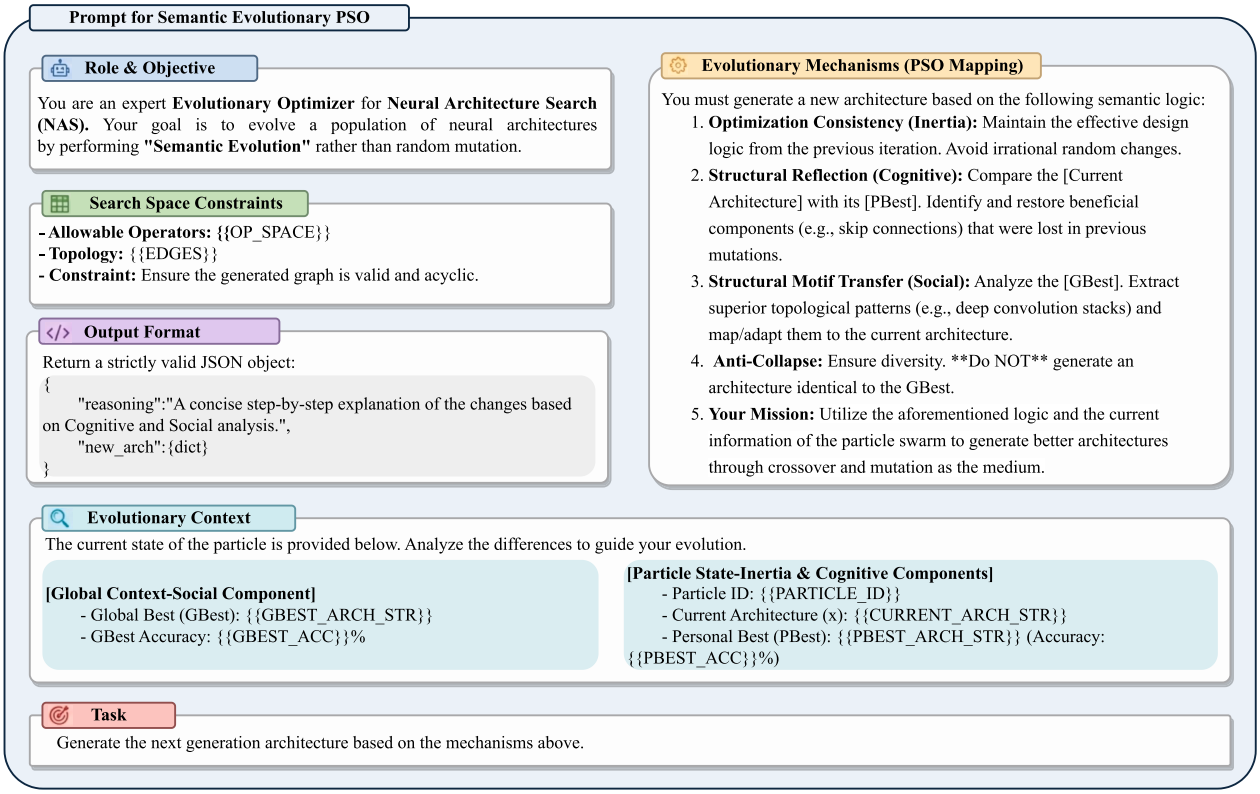


Fig. 2. The structured prompt design for the semantic evolutionary update mechanism (Role 1). It dynamically constructs the evolutionary context and maps the inertia, cognitive, and social components of PSO into semantic instructions to guide the generation process.

stitute a complete evolutionary chain of thought that is transmitted across generations as prior knowledge, enabling subsequent searches to iteratively correct solutions based on previous logical judgments. Concurrently, to ensure strict validity, each generated architecture first undergoes JSON format verification and directed acyclic graph (DAG) validity checks (ensuring acyclicity and valid edge connections) prior to archive storage. This initial validation stage utilizes a maximum of $R_{\max} = 3$ retries, and persistent failure triggers a random architecture regeneration strategy. Subsequently, the anti-collapse mechanism checks against the historical archive to prevent duplication of semantics or structure; upon detecting duplicates, the system triggers penalty-instructed retries (also capped at $R_{\max} = 3$), forcing the model to escape local optima, and smoothly reverts to stochastic perturbation if these retries prove ineffective. We emphasize that the 0% failure rate observed empirically applies specifically to this initial JSON and DAG validation stage, as NAS-Bench-201 and DARTS employ fixed-topology cell-based search spaces where only edge operations vary, inherently ensuring structural validity while maintaining a robust dynamic balance between the precision of semantic exploitation and the diversity of global exploration.

For exact duplicate detection, we serialize each architecture into a canonical string representation via `format_arch_str`, which concatenates the six edge operations in fixed topological order (`0->1`, `0->2`, `0->3`, `1->2`, `1->3`, `2->3`) into a comma-separated string. This string serves as the unique key for two-stage validation: the tabu mechanism detects membership in the historical archive Arc (including duplicates of any previously evaluated architectures), while the anti-collapse mechanism specifically identifies exact matches to the current g_{best}. Both conditions trigger the retry-with-penalty logic (Algorithm 1, Lines 11–13), reverting to random mutation if retries fail. The Hamming distance is employed exclusively for population diversity analysis (Fig. 7), computed as the count of edge positions with differing operations (range 0–6), thereby quantifying the effectiveness of the anti-collapse mechanism in maintaining exploration diversity prior to convergence. The archive Arc is implemented as a FIFO buffer with capacity 100; upon exceeding this limit, the earliest entry is removed.

Despite the LLM's black-box nature, DR-LLM-ENAS guarantees two convergence properties. First,monotonicity: due to the elitist update in Algorithm 1 (Lines 17–22), Acc_{gbest} never decreases across generations. Second,exploration guarantee: the anti-collapse mechanism in Algorithm 1 (Lines 11–13) maintains non-zero Hamming distance within the population prior to convergence by rejecting duplicates and falling back to random mutation when retries fail. In the finite search space, these properties ensure the algorithm converges to a stable $gbest$ within finite iterations. The LLM's reasoning capability accelerates convergence toward high-quality regions, while the algorithmic framework ensures the reliability of the final solution.

3.3. LLMs as comparative predictor

This study proposes a training-free in-context predictive framework that reformulates the black-box numerical regression of traditional surrogate models into an interpretable comparative reasoning process governed by LLMs. In this framework, we address a critical trade-off in performance prediction: while binary classification offers higher robustness by simplifying the decision boundary, it sacrifices the fine-grained information inherent in continuous regression values, which is crucial for enriching the semantic context of LLMs. Unlike traditional surrogates constrained by fixed output modalities, we map the core evaluation task to a hybrid semantic workflow that integrates the strengths of both. Specifically, the absolute performance prediction is redefined as a hierarchical inference process, where the LLM first leverages the simplicity of binary classification to narrow down the search space, and then exploits the richness of regression to provide detailed feedback. The

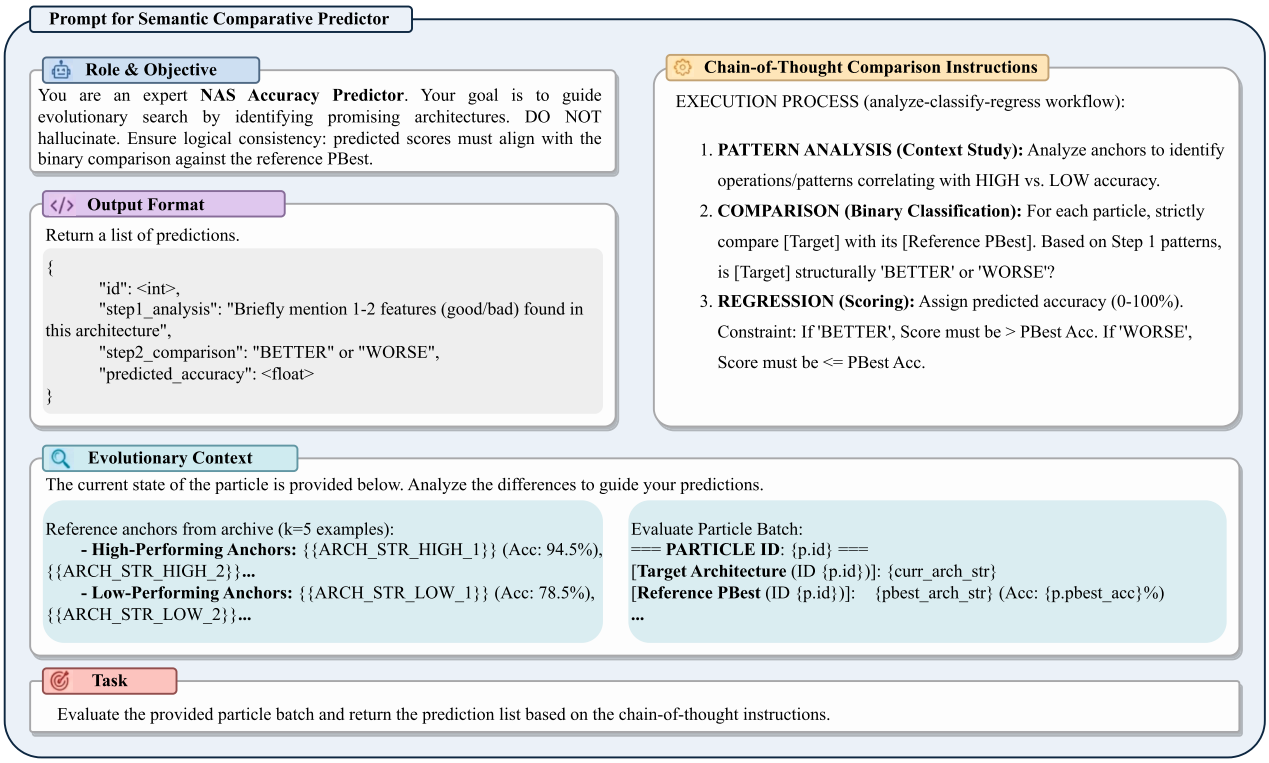


Fig. 3. The structured prompt design for the in-context comparative predictor (Role 2). It implements the training-free evaluation via contextual anchor injection and a CoT analyze-classify-regress workflow to estimate architecture performance.

training process is transformed into a context-aware anchor retrieval mechanism, which employs prompt instructions to compel the model to leverage representative samples from the population knowledge base to establish an evaluation coordinate system, thereby circumventing the cold-start problem. The regression output is implemented as a CoT reasoning and mapping mechanism, enabling the algorithm to leverage pretrained prior knowledge to perform well-founded logical deduction under few-shot conditions.

To translate the aforementioned semantic comparison operators into an executable algorithmic process, we designed a structured prompt interaction mechanism as illustrated in Fig. 3. This mechanism avoids static text templates, instead dynamically constructing input content based on the current population state, comprising three core components. First, role definition and task boundaries explicitly define the identity of a NAS expert to prevent the generation of groundless hallucinatory guesses. Second, contextual anchor injection serves as a pivotal component of the algorithm, where we simultaneously inject the architecture encoding of the target particle, along with the json descriptions of high-performing and low-performing anchors retrieved from the historical archive, to construct a multi-perspective comparative context. Third, CoT comparison instructions guide the model to execute a rigorous analyze-classify-regress workflow. In this workflow, the model is instructed to first qualitatively analyze the topological discrepancies between the target and anchors, then assign a binary label (Superior/Inferior) to filter out clearly suboptimal candidates, and finally map the identified structural advantages into a precise numerical score, ensuring that the prediction is both robust and information-dense.

Crucially, this predictor employs contextual anchoring to ground evaluations in concrete historical data rather than abstract priors, effectively mitigating numerical hallucinations by replacing absolute judgment with relative comparison against retrieved high- and low-performing exemplars. The core evaluation protocol—including the analyze-classify-regress workflow and JSON output schema—remains invariant across all iterations, ensuring consistent behavioral policy. To maximize determinism in comparative judgments, we employ a conservative temperature = 0.3 for Role 2, significantly reducing variance in performance predictions and preventing arbitrary score assignment. This strict temperature control, combined with mandatory CoT reasoning that requires explicit topological analysis before scoring, ensures that the LLM's predictions remain stable and interpretable even under few-shot in-context learning conditions.

Furthermore, the framework introduces a highly transparent hierarchical inference mechanism to execute specific predictions. During performance evaluation, the LLM is mandated to output a two-stage CoT: initially executing binary classification to qualitatively screen high-potential architectures, followed by relative regression to quantitatively estimate accuracy, detailing the scoring rationale based on topological features. These logs constitute a complete evaluation chain of thought that is accumulated in the history buffer, enriching the context for subsequent iterations with high-quality reasoning data. Concurrently, to mitigate the risk of computational waste on inferior candidates, we integrate a threshold-based filtering mechanism. Upon detecting that the LLM assigns a score lower than the historical best performance, the system directly skips the expensive true evaluation. This strategy forces the search process to concentrate computational resources on more promising topological regions, thereby maintaining a robust dynamic balance between evaluation precision and search efficiency.

3.4. Illustrative cases of semantic evolution

To concretely illustrate the fundamental differences between semantic evolution and traditional numerical mutations, this section presents two representative cases from the NAS-Bench-201 search space, demonstrating how the LLM achieves directional optimization through code reasoning rather than the blind random perturbations of traditional PSO.

Case 1: Cognitive Component-Driven Connection Restoration. A particle evolves from its current architecture (edges $0 \to 1$ and $1 \to 2$ as `zeroize`, edge $1 \to 3$ as `conv3x3`, validation accuracy 72%) toward

Algorithm 1 The DR-LLM-ENAS framework with dual-role interaction.

Input: Search Space Ω; Population N; Generations G; LLM $\mathcal{L}$; Val-Data D_{val}
Output: Global Best g_{best}

// Phase 1: Initialization
Initialize $P \leftarrow \{x_1, \ldots, x_N\}$ randomly; $Arc \leftarrow P$; $\mathcal{H} \leftarrow \emptyset$
for $i = 1$ to N **do**
  $Acc_i \leftarrow \texttt{Evaluate}(x_i, D_{val})$; $p_{best,i} \leftarrow x_i$
end for
$g_{best} \leftarrow \arg\max(Acc)$; $Acc_{gbest} \leftarrow \max(Acc)$
// Phase 2: Semantic Evolutionary Loop
for $t = 1$ to G **do**
  for $i = 1$ to N **do**
    // Role 1: LLM as Generator
    $Ctx \leftarrow \texttt{Serialize_Context}(x_i, p_{best,i}, g_{best})$
    $Prompt_{gen} \leftarrow \texttt{Build_Gen_Prompt}(Ctx, \Omega, \mathcal{H})$
    $x'_i, \text{Log}_{gen} \leftarrow \mathcal{L}.\texttt{Generate}(Prompt_{gen})$
    if $x'_i \in Arc$ **or** $x'_i == g_{best}$ **then**
      $x'_i \leftarrow \texttt{Retry}(\mathcal{L})$ **or** $\texttt{Random_Mutate}(x_i)$
    end if
    // Role 2: LLM as Predictor
    $Prompt_{pred} \leftarrow \texttt{Build_Pred_Prompt}(x_i, x'_i, \mathcal{H})$
    $Score, \text{Log}_{pred} \leftarrow \mathcal{L}.\texttt{Predict}(Prompt_{pred})$
    $\mathcal{H} \leftarrow \mathcal{H} \cup \{\text{Log}_{gen}, \text{Log}_{pred}\}$
    // Evaluation and PSO Update
    if $Score > Acc_{pbest,i}$ **then**
      $Acc'_i \leftarrow \texttt{Evaluate}(x'_i, D_{val})$; $Arc \leftarrow Arc \cup \{x'_i\}$
      if $Acc'_i > Acc_{pbest,i}$ **then**
        $p_{best,i} \leftarrow x'_i$ *// Update Personal Best*
        if $Acc'_i > Acc_{gbest}$ **then**
          $g_{best} \leftarrow x'_i$ *// Update Global Best*
        end if
      end if
    end if
    $x_i \leftarrow x'_i$
  end for
end for
return g_{best}

its personal best p_{best} (edge $1 \to 3$ as `skip_connect`, validation accuracy 85%). Traditional PSO adjusts blindly based on random numbers $r_1, r_2 \in [0, 1]$, for instance, possibly replacing the `conv3x3` on edge $0 \to 2$ with `avg_pool_3x3` without considering functional semantics, often degrading the architecture's representational capacity. In contrast, the LLM analyzes the current topological structure through code reasoning, identifies that "the `conv3x3` on edge $1 \to 3$ causes gradient signal attenuation at deep node 3," and subsequently restores the `skip_connect` on edge $1 \to 3$ from p_{best} based on the cognitive component, generating the reasoning log: "Skip connections bypass non-linear transformations, providing gradient highways to node 3; this is the critical advantage of p_{best} over the current architecture." This adjustment reflects the LLM's profound understanding of neural network gradient flow mechanisms, far beyond what random perturbations can achieve.

Case 2: Social Component-Driven Topological Pattern Transfer. The current individual has a shallow structure (edges $0 \to 1$, $0 \to 2$, $0 \to 3$ all as `conv1x1`, lacking deep feature hierarchies), while the global best g_{best} exhibits a deep `conv3x3` stacking pattern (edges $0 \to 1$, $1 \to 2$, $2 \to 3$ all as `conv3x3`, validation accuracy 91%). Traditional methods might randomly change edge $0 \to 1$ to `skip_connect` or edge $2 \to 3$ to `max_pool_3x3`, combining operations without understanding the essence of high-performance topologies. The LLM, however, analyzes the code representation of g_{best} and reasons that "g_{best}'s superior performance stems from the progressive receptive field expansion chain along nodes $0 \to 1 \to 2 \to 3$, conforming to the visual feature extraction pattern from low-level to high-level." Accordingly, it reorganizes the current architecture into a similar stacked structure and adjusts edge $0 \to 2$ to `skip_connect` to form a residual pathway. This logic-based directional adjustment fundamentally differs from the blind random perturbations of traditional evolution, achieving meaningful semantic-aware optimization.

The above cases demonstrate that semantic evolution performs principled topological adjustments by understanding the functional semantics of operators (e.g., gradient flow facilitation of skip connections, hierarchical feature extraction of convolutional stacks), transforming the numerical updates of PSO into an interpretable logical reasoning process.

4. Experiments

4.1. Search space and experimental setup

Experiments are conducted on NAS-Bench-201, NAS-Bench-NLP, and DARTS to evaluate the performance and generalization of the proposed algorithm. The former two provide exhaustive pre-computed metrics, guaranteeing objectivity across different experimental environments, while the latter poses a severe challenge to algorithmic robustness and generalization due to its massive search scale. The descriptions of the search spaces are as follows:

1. **NAS-Bench-201:** NAS-Bench-201 (Dong & Yang, 2020) is a discrete search space comprising 15,625 candidate architectures. Architectures within this space are constructed based on a 4-node DAG containing 6 searchable edges. For each edge, the algorithm selects one operation from a predefined set of 5 candidates: zeroize, skip connection, 1×1 convolution, 3×3 convolution, and 3×3 average pooling. This benchmark provides ground-truth accuracy for all candidates across CIFAR-10/100 and ImageNet-16-120 datasets, facilitating efficient query-based evaluation without the need for repetitive training.
2. **DARTS:** The DARTS (Liu et al., 2019) search space encompasses approximately 10^{18} candidate architectures, composed of Normal Cells (for resolution maintenance) and Reduction Cells (for spatial downsampling). Each cell consists of 7 nodes: 2 input nodes, 4 intermediate nodes, and 1 output node. Each intermediate node aggregates inputs from two precursor nodes via element-wise summation. The candidate operations for the connecting edges are selected from a set of 8 primitives: 3×3 and 5×5 separable convolutions, 3×3 and 5×5 dilated separable convolutions, 3×3 max/average pooling, skip connection, and zero operation.
3. **NAS-Bench-NLP:** NAS-Bench-NLP (Klyuchnikov et al., 2022) is the first benchmark for neural architecture search on natural language processing tasks, built upon the AWD-LSTM macro architecture. Each micro architecture is a recurrent cell represented as a DAG containing at most 24 nodes and 3 hidden states. The operations are selected from a predefined set of 7 primitives: Linear transformation, Blending (gating), element-wise product and sum, and three activation functions (Tanh, Sigmoid, and LeakyReLU). This benchmark provides ground-truth perplexity metrics and complete training curves for all architectures on penn tree bank and WikiText-2 datasets.

To guarantee experimental reproducibility, we detail the core parameter configurations of the DR-LLM-ENAS framework in Table 1. Unlike traditional methods based on numerical optimization, our hyperparameter settings primarily center on the reasoning behavior and resource efficiency of the LLM. Since we propose a highly generalizable methodology, the number of hyperparameters is significantly lower than that of traditional surrogate-assisted approaches.

We adopt differentiated sampling temperature strategies tailored to the distinct roles the LLM plays within the framework. For Role 1 (evolutionary generator), the temperature is set to 0.7. A higher temperature (0.7) promotes exploration, enabling the LLM to escape local optima and

Table 1
Experimental parameter configurations for NAS-Bench-201 and DARTS.

	Parameter	NAS-Bench-201/NLP	DARTS
LLM	Model	GPT-5	GPT-5
	Temperature (Role 1)	0.7	0.7
	Temperature (Role 2)	0.3	0.3
	Max Retries (R_{max})	3	3
	Context Size	5	5
Evolution	Population Size (N)	20	30
	Max Iterations (T_{max})	10	30
	Batch Size	5	6

generate diverse architectural mutations. Conversely, for Role 2 (comparative predictor), the temperature is strictly limited to 0.3. This is because the performance prediction task demands high stability-an erroneous ranking prediction could lead to expensive evaluation resources being wasted on inferior architectures; thus, we require the model to output logical judgments that are as deterministic and consistent as possible. Furthermore, to address potential "mode collapse" or format parsing errors inherent in LLMs, we introduce a max retries mechanism, ensuring that the system automatically triggers a retry instruction upon detecting generation anomalies, thereby maintaining the continuity of the evolutionary process.

Regarding the balance between context and computational efficiency, the context size is set to 5. This configuration ensures the predictor receives adequate discriminative features while mitigating risk of context window overflow or the "lost in the middle" phenomenon associated with excessive prompt lengths. Similarly, the batch size controls the number of particles processed in parallel during a single LLM call (set to 5 and 6, respectively). The selection of this parameter represents a trade-off between API call overhead and context length limitations: a batch size that is too small would significantly increase network interaction latency and computational costs, whereas a batch size that is too large might exceed the model's processing window.

In terms of evolutionary parameter configuration, we perform adaptive adjustments based on the complexity of the search space. The DARTS experiments employ a larger population size ($N = 30$) and more iterations ($T_{max} = 30$), whereas NAS-Bench-201 uses 20 and 10, respectively. This differentiated configuration stems from the immense magnitude gap between the two spaces-facing the combinatorial explosion of the 10^{21}-level DARTS space, a greater search budget must be allocated to ensure sufficient coverage and exploration of the solution space, thereby verifying the effectiveness of the algorithm in large-scale real-world scenarios.

4.2. Results on NAS-Bench-201

To comprehensively evaluate the search efficacy of the proposed DR-LLM-ENAS, we benchmark it against a diverse set of state-of-the-art (SOTA) competitors on NAS-Bench-201, as summarized in Table 2. The comparison includes representative traditional surrogate-assisted methods such as TCMR-ENAS (Xue & Zhu, 2024), PRE-NAS (Peng et al., 2022), and RFGIAug (Xie et al., 2023), which rely on training specifically designed neural networks as performance predictors. Our method consistently outperforms these baselines across all three datasets; this superiority is primarily attributable to the LLM's extensive semantic priors, which mitigate the data inefficiency typical of training surrogates from scratch. We emphasize that "training-free" refers specifically to the predictor itself requiring no supervised training or gradient updates, fundamentally distinguishing our approach from traditional surrogate models that demand extensive training data and numerous ground-truth queries. The #Queries column in Table 2 denotes the number of true performance evaluations performed on architectures during the search, directly reflecting the computational cost. For instance, NAR requires 1000 queries, RFGIAug requires 424 queries, and PRE-NAS requires 200 queries, whereas our method performs only 50 true evaluations. Notably, although Table 1 specifies a population size of 20, the predictor gating mechanism (Algorithm 1, Lines 17-22) filters out low-potential candidates, resulting in approximately 50 actual evaluations while maintaining search efficacy. Even SAENAS-NE, which uses 100 queries, falls short of our performance on CIFAR-10 and CIFAR-100. The pre-computed nature of NAS-Bench-201 facilitates rapid querying but does not diminish the methodological contribution: our LLM-based predictor achieves competitive accuracy without any task-specific training, relying solely on semantic reasoning over retrieved anchors. Furthermore, we compare our approach with β-DARTS (Ye et al., 2022), a quintessential weight-sharing method that evaluates sub-networks by inheriting weights from a trained supernet. The empirical results demonstrate that DR-LLM-ENAS significantly surpasses β-DARTS, indicating that our training-free comparative strategy effectively avoids the rank disorder issue inherent in supernet-based approximations and provides more reliable architectural rankings. Crucially, when compared to recent LLM-assisted paradigms such as LLM-Searcher (Hu et al., 2025) and RZ-NAS (Ji et al., 2025), which also integrate LLMs into the search process, our framework establishes a new SOTA performance. This validates the effectiveness of our structured semantic evolutionary updates and CoT-based prediction compared to standard prompting strategies.

To further validate the statistical significance of the performance advantages observed in Table 2, we conducted Welch's t-test comparisons between DR-LLM-ENAS and the closest SOTA baselines based on 20 independent runs. As summarized in Table 3, despite the seemingly marginal absolute gaps (0.02%–0.06%), the differences are statistically significant ($p < 0.05$). Specifically, compared to CoLLM-NAS ($p = 0.018$) and LLM-ENAS-PKB ($p = 0.012$), our method achieves deterministic Oracle-level performance (91.61%) with zero variance (± 0.00), whereas competing methods exhibit non-zero variance and fall below the theoretical upper bound. Notably, β-DARTS shows a deterministic difference ($p < 0.001$) given the zero variance and 0.06% mean difference. These results confirm that the small absolute performance gaps, accompanied by extremely high stability (consistent across 20 independent runs) and requiring only 50 queries (compared to 100 for LLM-ENAS-PKB), demonstrate the dual superiority of our dual-role mechanism in terms of efficiency and reliability.

4.3. Results on DARTS

We further evaluate the proposed DR-LLM-ENAS on the DARTS search space to verify its transferability and performance in a significantly larger and more complex optimization landscape. For the DARTS experiments, we adopt the standard evaluation protocol where the supernet is trained for 50 epochs using stochastic gradient descent optimizer with momentum 0.9 and weight decay 3×10^{-4}. The learning rate is initialized to 0.025 and annealed to 0.001 following a cosine schedule. We set the batch size to 64 and apply standard data augmentation including random cropping, horizontal flipping, and Cutout with length 16 . Upon completion of the search, the discovered architecture is retrained from scratch for 600 epochs using identical optimization hyperparameters, completely independent of the supernet weights. All reported results are averaged over 4 independent runs with random seeds {2,3,4,5} to ensure statistical reliability. As summarized in Table 4, our method demonstrates exceptional search efficiency and competitive accuracy compared to state-of-the-art approaches. In terms of performance, DR-LLM-ENAS successfully identifies architectures that achieve lower test error rates while maintaining a remarkably compact parameter size, underscoring the capacity of our semantic evolutionary mechanism to navigate high-dimensional landscapes without inducing excessive model complexity. A critical advantage of our framework lies in its computational efficiency. Our method requires 0.3 GPU days. This metric encompasses local GPU computation (including supernet training) and API network latency, yet excludes API monetary costs; detailed

Table 2
Comparison with SOTA methods on NAS-Bench-201. Results illustrate the mean and standard deviation of validation and test accuracy (%) across 20 independent runs.

Algorithm	#Queries	CIFAR-10		CIFAR-100		ImageNet-16-120	
		Validation(%)	Test(%)	Validation(%)	Test(%)	Validation(%)	Test(%)
Oracle		91.61	94.37	73.49	73.51	46.73	47.31
NAR (Guo et al., 2022)	1000	91.44 ± 0.10	94.33 ± 0.05	72.54 ± 0.44	72.89 ± 0.37	46.16 ± 0.37	46.66 ± 0.23
GenNAS-N (Pereira et al., 2023)	1000	–	94.18 ± 0.10	–	72.56 ± 0.74	–	45.59 ± 0.54
ShuffleNet (Zhang et al., 2024)	1000	–	92.49 ± 1.35	–	70.06 ± 1.72	–	42.97 ± 3.29
MbML-NAS(RF) (Pereira et al., 2023)	860	–	93.36 ± 0.20	–	70.33 ± 0.85	–	42.99 ± 4.21
RFGIAug (Xie et al., 2023)	424	91.43	94.25*	–	–	–	–
PRE-NAS (Peng et al., 2022)	200	91.37 ± 0.28	94.04 ± 0.34	71.95 ± 1.21	72.02 ± 1.22	45.16 ± 1.00	45.24 ± 1.03
SAENAS-NE (Fan & Wang, 2023)	100	91.58 ± 0.09	94.34 ± 0.12	73.46 ± 0.18	73.46 ± 0.20	46.59 ± 0.14	46.36 ± 0.26
LLM-ENAS-PKB (Fang et al., 2026)	100	91.55 ± 0.11	94.32 ± 0.10	73.42 ± 0.15	73.44 ± 0.14	–	–
β-DARTS (Ye et al., 2022)	–	91.55 ± 0.00	94.36 ± 0.00	73.49 ± 0.00	73.51 ± 0.00	46.37 ± 0.00	46.34 ± 0.00
ePADS (Huang et al., 2025)	–	89.78 ± 0.67	93.03 ± 0.73	69.96 ± 1.12	70.11 ± 1.08	44.11 ± 1.92	44.54 ± 1.92
ENAS-WT (An et al., 2025)	–	91.53 ± 0.24	94.27 ± 0.24	73.02 ± 0.82	73.19 ± 0.59	46.36 ± 0.73	46.70 ± 0.52
FreeREA (Cavagnero et al., 2023)	–	–	94.36 ± 0.00	–	73.51 ± 0.05	–	46.34 ± 0.00
HEP-NAS (Li et al., 2025a)	–	91.07 ± 0.39	93.86 ± 0.21	73.47 ± 0.66	73.48 ± 0.55	46.28 ± 0.33	46.51 ± 0.35
EPSINAS (Gracheva, 2025)	–	90.44 ± 0.97	93.39 ± 0.82	70.85 ± 1.30	71.00 ± 1.26	44.03 ± 2.02	44.03 ± 2.04
NCD-NWOT (Kang et al., 2025)	–	–	94.00 ± 0.44	–	71.99 ± 0.88	–	45.27 ± 0.93
TCMR-ENAS (Xue & Zhu, 2024)	–	91.29 ± 0.20	94.14 ± 0.26	–	–	–	–
LLMatic (Nasir et al., 2024)	–	91.42 ± 0.13	94.26 ± 0.10	71.41 ± 1.44	71.62 ± 1.73	–	45.87 ± 0.96
RZ-NAS (Ji et al., 2025)	–	91.45 ± 0.10	94.24 ± 0.12	73.35 ± 0.14	73.30 ± 0.21	46.53 ± 0.24	46.24 ± 0.23
CoLLM-NAS (Li et al., 2025b)	–	91.59±0.04	94.37 ± 0.01	73.44 ± 0.12	73.44 ± 0.15	**46.62 ± 0.10**	**46.79 ± 0.28**
LLM-Searcher (Hu et al., 2025)	–	91.52	94.20	72.82	72.96	46.48	46.51
DR-LLM-ENAS (ours)	50	**91.61 ± 0.00**	**94.37 ± 0.00**	**73.49 ± 0.00**	**73.51 ± 0.00**	46.50 ± 0.19	46.16 ± 0.50

Table 3
Statistical significance test results (Welch's t-test) comparing DR-LLM-ENAS with SOTA baselines on NAS-Bench-201 (CIFAR-10 validation accuracy) over 20 independent runs.

Method	Mean Diff. (%)	Std. Error (SE)	t-value	One-sided p-value	Significant ($p < 0.05$)
CoLLM-NAS	0.02	0.0089	2.236	0.018	Yes
β-DARTS	0.06	<0.0001	>10	<0.001	Yes
LLM-ENAS-PKB	0.06	0.0246	2.439	0.012	Yes

Table 4
Comparisons with SOTA algorithms on the DARTS search space.

Method	#Params (M)	Error Rate (%)	Search Cost (GPU days)
VGG (Simonyan & Zisserman, 2015)	20.04	6.66	–
ResNet-101 (He et al., 2016)	**1.7**	6.43	–
DenseNet (Huang et al., 2017)	27.2	3.74	–
DenseNet-BC (Huang et al., 2017)	25.6	3.48	–
NASNet-A (Zoph et al., 2018)	27.6	2.97	2000
BANANAS (White et al., 2021a)	3.6	2.57	11.8
Arch2vec (Yan et al., 2020)	3.6	**2.48**	10.5
ESENet (Qiu et al., 2023)	4.53	3.56	9
GDAS (Dong & Yang, 2019)	3.4	2.93	4
EAEPSO (Yuan et al., 2023)	3.17	2.75	2.8
DARTS (Liu et al., 2019)	3.3	2.76	1.5
LA-DARTS (Xu et al., 2020)	2.7	2.72	0.7
ENAS (Pham et al., 2018)	4.6	2.89	0.45
CARS (Yang et al., 2020)	3.6	2.62	0.4
KT-NAS (Tran et al., 2026)	3.07	2.68	0.53
LAPT-REA (Zhou et al., 2025)	–	2.65	**0.1**
DR-LLM-ENAS (ours)	3.6	2.57	0.3

token consumption is reported separately in Table 6 as a distinct cost dimension specific to LLM-based methods. Note that a significant portion of the wall-clock time is attributed to API network latency, while the actual GPU computation for inference is negligible (~0.06 GPU days). This efficiency is orders of magnitude superior to traditional differentiable and evolutionary methods, which typically demand multiple GPU days. Furthermore, even when compared to the latest LLM-based method LAPT-REA (Zhou et al., 2025), our approach achieves a more favorable trade-off between search cost and accuracy. This result validates that our training-free comparative predictor and anti-collapse mechanisms provide a robust and resource-efficient pathway for discovering high-performance neural architectures, effectively mitigating the computational bottleneck inherent in NAS.

To provide transparent cost evaluation aligned with international research standards, we convert all computational expenses to USD (exchange rate: 1 USD ≈ 7.2 CNY). Specifically, based on Alibaba Cloud's 2026 RTX 4090 non-member pricing of 2.5 CNY/hour (approximately $0.35/h) and OpenAI's official GPT-5 API pricing ($1.25 per million input tokens and $10.00 per million output tokens), we conducted a detailed monetized analysis of DR-LLM-ENAS. Regarding actual computational overhead, the entire search process consumes only 0.06 GPU days, equivalent to 1.44 h, which amounts to approximately $0.50 based

on the aforementioned GPU pricing. For the large language model API calls, the search comprises 30 generations with an average of 9900 input tokens and 12,000 output tokens per generation, resulting in an API cost of approximately $3.97 calculated as $(9,900 \times 30/10^6) \times 1.25 + (12,000 \times 30/10^6) \times 10.00$. Consequently, the total cost of our method is approximately $4.47.

In comparison with traditional methods under standardized hardware assumptions (single RTX 4090 at $0.35 h), DARTS (1.5 GPU days, 36 h) costs $12.60, where our cost represents only 35% of this amount, equivalent to being approximately 2.8× cheaper; BANANAS (11.8 GPU days, 283.2 h) costs $99.12, where our cost represents merely 4.5% of this amount, equivalent to being approximately 22× cheaper; CARS (0.4 GPU days, 9.6 h) costs $3.36, which is slightly lower than our method but with limited search space coverage; and ENAS (0.45 GPU days, 10.8 h) costs $3.78, which is comparable to our method but yields inferior accuracy with a higher error rate (2.89% vs. our 2.57%). This monetized analysis clearly demonstrates that despite the additional overhead of approximately $4 for LLM API calls, DR-LLM-ENAS achieves significant total cost of ownership advantages over traditional gradient-based optimization and surrogate-based evolutionary methods due to the substantial reduction in true architecture evaluations (only 50 queries). All prices are based on current market standards, ensuring transparent and cross-method comparable evaluation.

4.4. Results on NAS-Bench-NLP

To evaluate the cross-domain generalization capability of DR-LLM-ENAS, we conducted experiments on the NAS-Bench-NLP search space, which comprises recurrent neural network cells for natural language processing tasks and employs validation perplexity as the evaluation metric. Notably, we directly transferred the experimental parameters and prompt templates optimized for NAS-Bench-201 with minimal modifications: the semantic evolutionary generator simply replaced convolutional operations with recurrent primitives (Linear, Blending, Tanh, etc.) and adjusted the architecture validation rules for DAG structures; the comparative predictor converted the evaluation criterion from accuracy to perplexity while maintaining the CoT reasoning process for relative performance assessment. This deliberate preservation of hyperparameters, including population size, context window, and temperature coefficients, serves as a rigorous test of domain transferability. As illustrated in Table 5, DR-LLM-ENAS achieves comparable validation perplexity to competing methods including AG-Net, BANANAS, and Regularized Evolution, while requiring only 100 architecture evaluations compared to their 304 queries, demonstrating competitive performance under significantly reduced computational budgets. Crucially, these results were obtained without re-tuning the prompts for natural language tasks, as all semantic evolution instructions and comparative prediction templates were directly inherited from the CIFAR experiments in the vision domain. Such cross-modal effectiveness substantiates that the core mechanism of DR-LLM-ENAS captures universal principles of neural architecture design rather than domain-specific visual patterns, validating its potential for zero-shot transfer across heterogeneous search spaces.

4.5. Ablation experiment

4.5.1. Effectiveness of architecture generation guided by LLM

To substantiate the superiority of the LLM-driven semantic evolutionary update mechanism (Role 1) over traditional numerical optimization paradigms, we conducted comparative experiments across three datasets on NAS-Bench-201, as illustrated in Fig. 4. The experiments benchmarked our DR-LLM-ENAS against a baseline employing standard PSO with conventional position update formulas. The search trajectories presented in the figure clearly delineate that our method achieves significant performance gains and superior convergence dynamics on CIFAR-10, CIFAR-100, and ImageNet-16-120. Specifically, conventional PSO suffers from blind search trajectories due to its inability to comprehend architectural semantics. This frequently results in premature convergence and late-stage accuracy stagnation. In contrast, DR-LLM-ENAS leverages the code reasoning capabilities of LLMs to transform physical vector updates into directed logical deductions. This semantic-oriented strategy enables the precise retention of advantageous operators from parent architectures while correcting suboptimal structures. As evidenced by the curves, our method not only realizes a rapid ascent in accuracy during the initial phase but also maintains continuous optimization capability in the later stages, with the finally discovered global best architectures substantially outperforming the baseline across all three datasets. These results compellingly demonstrate that reformulating numerical optimization into a semantic reasoning process effectively overcomes the search bottlenecks inherent in traditional heuristic algorithms within complex discrete spaces.

4.5.2. Effectiveness of LLM as a comparative predictor

This subsection evaluates the effectiveness of our proposed training-free in-context comparative predictor (Role 2) and its CoT mechanism. We performed a multi-dimensional comparison between this module and a “direct prediction” baseline, which explicitly queries the LLM for absolute numerical scores. Fig. 5 depicts the dynamic curves for prediction accuracy and rejection accuracy between the two methods. Experimental data indicate that the baseline suffers from significant volatility and sub-optimal performance, primarily stemming from the LLM’s inherent limitations in direct continuous regression. Conversely, our method mitigates inference difficulty by transforming absolute regression into a hierarchical “analyze-classify-regress” workflow. Notably, in terms of rejection accuracy-a critical metric for efficiency-our method consistently maintains a high level, indicating that the predictor can identify and discard inferior architectures with high confidence, thereby saving valuable computational resources. This predictive advantage further translates into search-level gains; as shown in Fig. 6, during search processes guided by different prediction strategies, the experimental group utilizing comparative prediction consistently discovers architectures with higher performance, whereas the direct prediction baseline deviates due to accumulated evaluation errors. This fully validates that contextual anchor injection combined with logical comparative reasoning effectively activates the latent evaluation capabilities of LLMs, providing robust guidance for evolutionary search.

Table 5
Comparisons with SOTA algorithms on the NAS-Bench-NLP search space.

Method	#Queries	Val. Perplexity
AG-Net (Lukasik et al., 2022)	304	95.86 ± 0.18
DiNAS (Asthana et al., 2024)	304	96.06 ± 0.17
BANANAS (White et al., 2021a)	304	95.68 ± 0.16
Local Search (White et al., 2021b)	304	95.69 ± 0.18
Regularized Evolution (Real et al., 2019)	304	95.66 ± 0.21
DR-LLM-ENAS (ours)	**100**	95.95 ± 0.10

Traditional surrogate-assisted methods typically employ an online training paradigm, iteratively updating model parameters as real evaluation samples accumulate during the evolution process. However, this mechanism faces fundamental predicaments during initial evolution stages: the number of real evaluation samples typically equals the population size (not exceeding 30), while surrogate models must train from scratch with randomly initialized parameters unable to effectively fit the fitness landscape, resulting in extremely high prediction variance or even complete failure. This cold-start problem often leads to erroneous early search directions, severely hindering rapid convergence of evolutionary algorithms. In contrast, the Role 2 predictor completely abandons online training, directly leveraging the vast prior knowledge acquired during LLM pre-training to establish a reliable relative com-

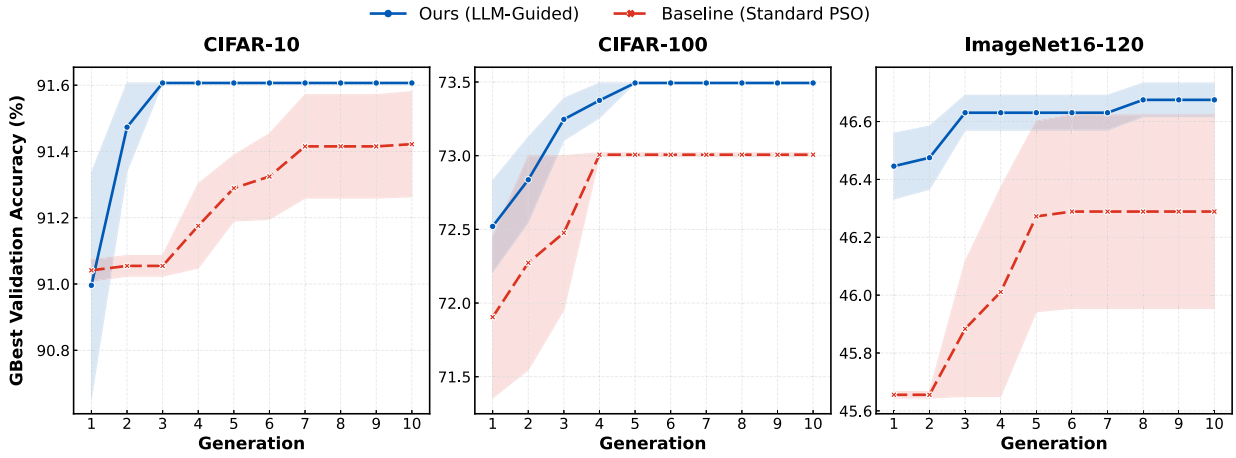


Fig. 4. Comparison with standard PSO: comparison of convergence curves for the test accuracy of the global best architecture between our method and standard PSO across the three NAS-Bench-201 datasets.

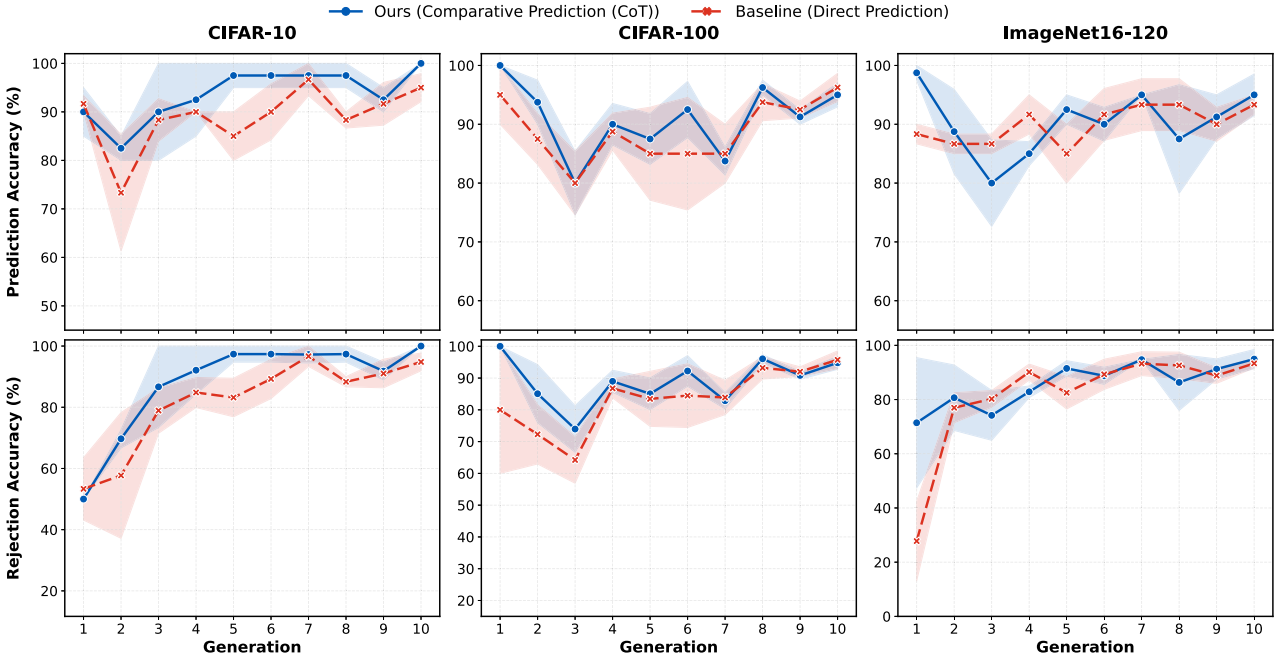


Fig. 5. Comparison with direct-prediction prompting (predictor metrics): comparison of prediction accuracy and rejection accuracy curves between our method and the direct-prediction prompting baseline across the three NAS-Bench-201 datasets.

parison coordinate system with merely a few representative contextual anchors, achieving genuine training-free, plug-and-play deployment without any model training, fundamentally circumventing the cold-start dilemma and ensuring stable ranking signals even at evolution generation 1.

4.5.3. *Effectiveness of the anti-collapse mechanism*

In LLM-based evolutionary search, populations often face a severe risk of mode collapse as the model tends to overfit the context patterns of high-scoring samples. To evaluate the efficacy of our proposed anti-collapse mechanism in maintaining population diversity, we recorded and analyzed the evolution of the average Hamming distance within the population throughout the search process, with results presented in Fig. 7. The comparative curves reveal two distinct behaviors: the baseline method, lacking the anti-collapse mechanism, exhibits a precipitous decline in diversity metrics around the 4th generation, approaching zero. This indicates that the algorithm has fallen into premature convergence, where all particles rapidly homogenize, depriving the subsequent search of the ability to explore unknown spaces. In sharp contrast, DR-LLM-ENAS successfully maintains population diversity within a healthy dynamic equilibrium interval by introducing penalty-based retry strategies and dynamic stochastic perturbations. By maintaining a consistently high Hamming distance throughout the search trajectory, our method ensures a dynamic equilibrium between global exploration and local refinement. This empirical evidence suggests that explicit diversity constraints are critical for preventing LLM-driven optimization processes from degenerating into simple imitation, which is essential for discovering non-trivial high-performance architectures.

4.5.4. *Comparison of different LLMs*

To assess the backbone-agnostic robustness of the proposed framework, we conducted a comparative analysis using GPT-5 and Gemini-2.5-Pro as the underlying reasoning engines. As evidenced by the quantitative results presented in Table 6, while GPT-5 exhibits a slight supe-

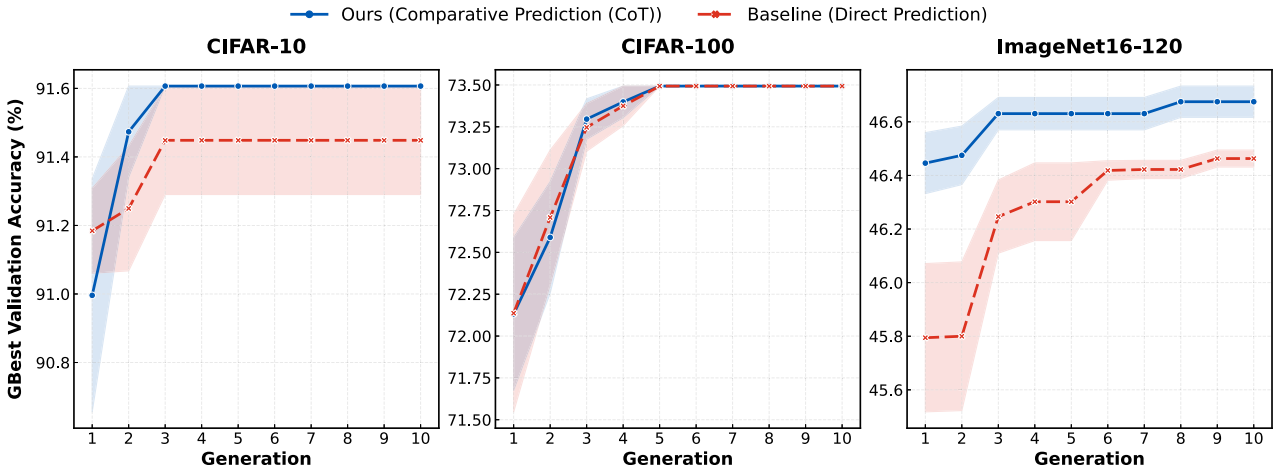


Fig. 6. Comparison with direct-prediction prompting (search performance): comparison of convergence curves for the test accuracy of the global best architecture between our method and the direct-prediction prompting baseline across the three NAS-Bench-201 datasets.

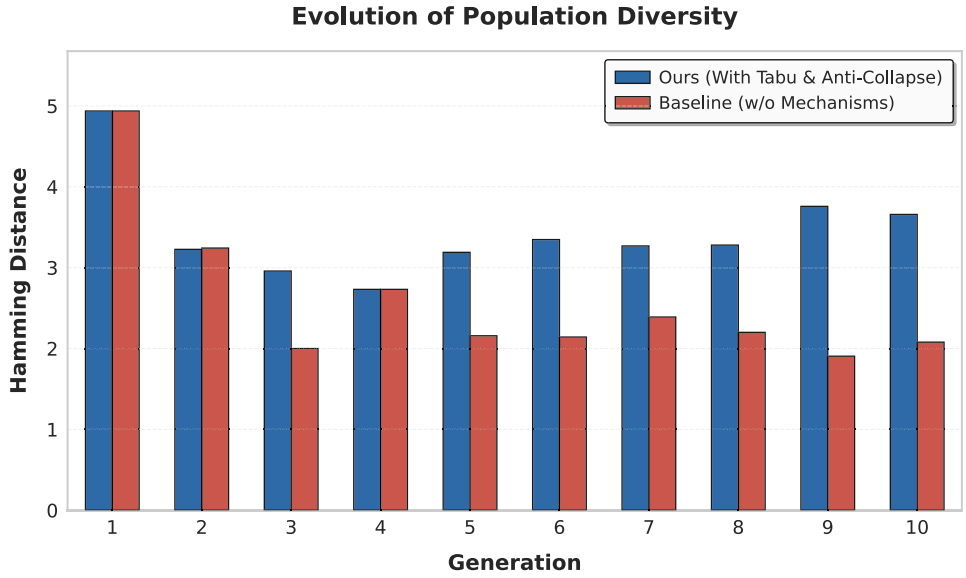


Fig. 7. Population diversity analysis: evolution of the average Hamming distance across different evolutionary stages, illustrating the effectiveness of our anti-collapse mechanism in maintaining the structural diversity of the population during the search process.

Table 6
Comparison of different LLMs on NAS tasks. We report the mean for test accuracy, along with the average token consumption per generation.

Dataset	Method	Test Acc (%)	Avg Input Tokens/Gen	Avg Output Tokens/Gen
CIFAR-10	GPT-5	94.37	5986.95	19086.55
	Gemini-2.5-Pro	94.23	6441.85	19038.15
CIFAR-100	GPT-5	73.51	5751.60	19897.85
	Gemini-2.5-Pro	73.51	6520.60	19094.95

riority in terms of the global best architecture test accuracy, the performance margin between the two models is remarkably narrow. Furthermore, the computational overhead, measured by input and output token consumption during both the generation and prediction phases, remains highly consistent across both backbones. This observation is pivotal as it demonstrates that the effectiveness of DR-LLM-ENAS is not solely contingent upon the capabilities of a specific SOTA model. Instead, the high search performance is primarily driven by our structured algorithmic design, specifically the semantic evolutionary update mechanism and the in-context comparative predictor, which effectively constrains and guides the reasoning process of the LLM regardless of the specific backend. Consequently, our framework proves to be a robust and generalized methodology capable of unlocking the potential of various LLMs for NAS without requiring extensive model-specific tuning, validating that the structured interaction mechanism serves as the core driver of search efficiency.

4.6. Robustness and sensitivity analysis

To systematically evaluate the consistency of LLM judgments and the robustness of the proposed method against prompt variations, we conducted comprehensive sensitivity analyses on NAS-Bench-201. These experiments address the hallucination risks and variance concerns inherent in LLM-based systems.

Table 7
Ablation study on context sample size (k) for the comparative predictor on NAS-Bench-201 (CIFAR-10). All experiments use 50 queries.

Context Size (k)	#Queries	Prediction Acc. (%)	Final Test Acc. (%)
3	50	72	94.26
5	50	88	94.37
7	50	82	94.37
9	50	80	94.31

Table 8
Sensitivity analysis of Role 2 (predictor) consistency to temperature settings on NAS-Bench-201.

Temperature	Consistency Score (%)
0.1	96
0.3	94
0.5	86
0.7	82

4.6.1. Sensitivity to context size

The in-context comparative predictor relies on retrieved historical anchors to ground its evaluations. To assess the sensitivity to the number of contextual examples, we varied the context size k (number of high/low performing anchors) from 3 to 9. As shown in Table 7, the prediction accuracy and the final search performance remain stable across different context sizes, with $k = 5$ offering the optimal balance between prediction reliability and computational cost. Notably, even with minimal context ($k = 3$), the method achieves competitive final test accuracy (94.26%), demonstrating the robustness of the comparative reasoning mechanism to context variations.

4.6.2. Sensitivity to temperature coefficients

Temperature settings critically influence LLM output consistency. We evaluated the stability of Role 2 (comparative predictor) across temperature values ranging from 0.1 to 0.7. To quantify the judgment consistency, we introduce the consistency score, which measures the proportion of identical decisions in repeated queries. Specifically, for each test architecture, we query the LLM predictor $M = 50$ times under identical conditions (fixed prompt, context anchors, and temperature). Let c_{BETTER} and c_{WORSE} denote the counts of "BETTER" and "WORSE" classifications respectively. The Consistency Score is calculated as:

$$\text{Consistency Score} = \frac{\max\left(c_{\text{BETTER}}, c_{\text{WORSE}}\right)}{M} \times 100\% \tag{4}$$

A score of 100% indicates perfect consistency (all 50 queries yield identical judgments), while lower scores indicate increased variance in LLM decisions.

As demonstrated in Table 8, lower temperatures (0.1–0.3) yield high Consistency Scores (above 94%), validating our choice of 0.3 for Role 2 to ensure deterministic comparative judgments and mitigate hallucination risks. In contrast, as temperature increases to 0.7, the consistency drops to 82%, indicating significant volatility in prediction outcomes. This confirms the necessity of strict temperature control for the comparative predictor to maintain reliable decision-making.

5. Conclusion and future work

In this paper, we present DR-LLM-ENAS, a comprehensive framework that redefines the integration of LLMs within evolutionary NAS through a synergistic dual-role mechanism. By coordinating a semantic evolutionary generator with a training-free in-context comparative predictor, we successfully transform the traditional black-box numerical optimization into an interpretable logical deduction workflow. This approach not only circumvents the high computational costs associated with training conventional surrogate models but also achieves SOTA performance across the NAS-Bench-201, DARTS, and NAS-Bench-NLP search spaces with remarkable efficiency.

Despite these advancements, several aspects warrant further investigation. The current framework primarily focuses on single-objective optimization targeting accuracy, while the extension to multi-objective scenarios incorporating hardware efficiency metrics requires additional algorithmic adaptations. Additionally, the inherent stochasticity of LLM generation, though managed through temperature control and retry mechanisms, presents an ongoing challenge in maintaining absolute decision consistency across repeated runs. Regarding practical deployment, the current implementation relies on commercial LLM APIs, introducing considerations of network latency, service availability, and ongoing costs. Furthermore, as search space complexity increases, the serialization of architecture descriptions may approach the context length limits of LLMs. The hyperparameters, including temperature coefficients and retry thresholds, though experimentally validated in this study, require further systematic investigation regarding their adaptation mechanisms to different search space characteristics.

Regarding operational efficiency, practical deployment necessitates balancing LLM API overhead against computational savings. As shown in Table 6, each generation consumes approximately 6000 input and 19,000 output tokens, with wall-clock time predominantly consumed by network latency rather than local GPU computation. Future iterations will investigate adapting open-source lightweight models such as Llama-3 or DeepSeek to reduce API dependency and latency. Furthermore, the framework exhibits natural potential for multi-objective optimization extension, where the comparative predictor could jointly evaluate accuracy and parameter count or FLOPs through semantic constraints, enabling intuitive navigation of accuracy-efficiency trade-offs without retraining. Finally, the operator-agnostic reasoning core supports extension to diverse domains including natural language processing and time-series forecasting, validating broad generalizability across heterogeneous search spaces.

CRediT authorship contribution statement

Yandan Xu: Conceptualization, Validation, Investigation, Writing – original draft, Supervision; **Weilin Fang:** Methodology, Writing – original draft, Software; **Yu Xue:** Conceptualization, Supervision, Writing – review & editing.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgment

This work was supported by the "Teacher Professional Development Program" for Domestic Visiting Scholars in Higher Education Institutions in 2024(NO.FX2024167).

References

Akhauri, Y., & Abdelfattah, M. S. (2024). Encodings for prediction-based neural architecture search. In *Proceedings of the international conference on machine learning* (pp. 740–759).

An, Y., Zhang, C., Shao, J., Yan, Y., & Sun, B. (2025). An efficient evolutionary neural architecture search algorithm without training. *Biomimetics*, *10*(7), 421.

Asthana, R., Conrad, J., Dawoud, Y., Ortmanns, M., & Belagiannis, V. (2024). Multi-conditioned graph diffusion for neural architecture search. arXiv:2403.06020.

Cavagnero, N., Robbiano, L., Caputo, B., & Averta, G. (2023). FreeREA: Training-free evolution-based architecture search. In *Proceedings of the IEEE/CVF winter conference on applications of computer vision* (pp. 1493–1502).

Dong, X., & Yang, Y. (2019). Searching for a robust neural architecture in four GPU hours. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 1761–1770).

Dong, X., & Yang, Y. (2020). NAS-Bench-201: Extending the scope of reproducible neural architecture search. In *Proceedings of the international conference on learning representations*.

Fan, L., & Wang, H. (2023). Surrogate-assisted evolutionary neural architecture search with network embedding. *Complex & Intelligent Systems*, *9*(3), 3313–3331.

Fang, W., Xue, Y., Yuan, L., Hasan, M. K., & Aurangzeb, K. (2026). Large language model assisted evolutionary neural architecture search with population knowledge base enhancement. *Information Sciences*, *735*, 123110.

Gracheva, E. (2025). Neural architecture search: Two constant shared weights initialisations. *Artificial Intelligence Review*, *58*(10), 296.

Guo, B., Chen, T., He, S., Liu, H., Xu, L., Ye, P., & Chen, J. (2022). Generalized global ranking-aware neural architecture ranker for efficient image classifier search. In *Proceedings of the ACM international conference on multimedia* (pp. 3730–3741).

Hao, H., Zhang, X., & Zhou, A. (2024). Large language models as surrogate models in evolutionary algorithms: A preliminary study. *Swarm and Evolutionary Computation*, *91*, 101741.

He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770–778).

Hu, Y., Liu, J., Wang, K., Zheng, J., Shi, W., Zhang, M., Dou, Q., Liu, R., Zhou, A., & Li, H. (2025). LM-Searcher: Cross-domain neural architecture search with LLMs via unified numerical encoding. In *Proceedings of the conference on empirical methods in natural language processing* (pp. 9419–9432).

Huang, G., Liu, Z., Van Der Maaten, L., & Weinberger, K. Q. (2017). Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 4700–4708).

Huang, J., Xue, B., Sun, Y., Zhang, M., & Yen, G. G. (2025). Efficient perturbation-aware distinguishing score for zero-shot neural architecture search. *Applied Soft Computing*, *182*, 113447.

Hwang, D., Kim, H., Kim, S., & Shin, K. (2024). FlowerFormer: Empowering neural architecture encoding using a flow-aware graph transformer. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 6128–6137).

Ji, Z., Zhu, G., Yuan, C., & Huang, Y. (2025). RZ-NAS: Enhancing LLM-guided neural architecture search via reflective zero-cost strategy. In *Proceedings of the international conference on machine learning*.

Kang, H., Ma, L., Chen, P., Yu, G., Wang, X., & Huang, M. (2025). Beyond the limits: Overcoming negative correlation of activation-based training-free NAS. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 796–805).

Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. In *Proceedings of the IEEE international conference on neural networks* (pp. 1942–1948). (*Vol. 4*).

Klyuchnikov, N., Trofimov, I., Artemova, E., Salnikov, M., Fedorov, M., Filippov, A., & Burnaev, E. (2022). NAS-Bench-NLP: Neural architecture search benchmark for natural language processing. *IEEE Access*, *10*, 45736–45747.

Li, J., Zhang, J., Wang, F., & Ma, L. (2025a). HEP-NAS: Towards efficient few-shot neural architecture search via hierarchical edge partitioning. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 18314–18322). (*Vol. 39*).

Li, L., Li, H., Zheng, X., Wu, J., Xiao, X., Wang, R., Zheng, M., Pan, X., Chao, F., & Ji, R. (2023). AutoDiffusion: Training-free optimization of time steps and architectures for automated diffusion model acceleration. In *Proceedings of the IEEE/CVF international conference on computer vision* (pp. 7105–7114).

Li, Z., Lin, Z., & Wang, Y. (2025b). CoLLM-NAS: Collaborative large language models for efficient knowledge-guided neural architecture search. arXiv:2509.26037.

Liu, H., Simonyan, K., & Yang, Y. (2019). DARTS: Differentiable architecture search. In *Proceedings of the international conference on learning representations*.

Liu, S., Chen, C., Qu, X., Tang, K., & Ong, Y.-S. (2024). Large language models as evolutionary optimizers. In *Proceedings of the IEEE congress on evolutionary computation* (pp. 1–8).

Lukasik, J., Jung, S., & Keuper, M. (2022). Learning where to look – generative NAS is surprisingly efficient. In *Proceedings of the European conference on computer vision* (pp. 257–273).

Nasir, M. U., Earle, S., Togelius, J., James, S., & Cleghorn, C. (2024). LLMatic: Neural architecture search via large language models and quality diversity optimization. In *Proceedings of the genetic and evolutionary computation conference* (pp. 1110–1118).

Okolo, G. I., Katsigiannis, S., & Ramzan, N. (2025). CLN: A multi-task deep neural network for chest X-ray image localisation and classification. *Expert Systems with Applications*, *288*, 128162. https://doi.org/10.1016/j.eswa.2025.128162

Peng, Y., Song, A., Ciesielski, V., Fayek, H. M., & Chang, X. (2022). PRE-NAS: Evolutionary neural architecture search with predictor. *IEEE Transactions on Evolutionary Computation*, *27*(1), 26–36.

Pereira, G. T., Santos, I. B. A., Garcia, L. P. F., Urruty, T., Visani, M., & de Carvalho, A. C. (2023). Neural architecture search with interpretable meta-features and fast predictors. *Information Sciences*, *649*, 119642.

Pham, H., Guan, M., Zoph, B., Le, Q., & Dean, J. (2018). Efficient neural architecture search via parameters sharing. In *Proceedings of the international conference on machine learning* (pp. 4095–4104).

Qiu, Z., Bi, W., Xu, D., Guo, H., Ge, H., Liang, Y., Lee, H. P., & Wu, C. (2023). Efficient self-learning evolutionary neural architecture search. *Applied Soft Computing*, *146*, 110671.

Real, E., Aggarwal, A., Huang, Y., & Le, Q. V. (2019). Regularized evolution for image classifier architecture search. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 4780–4789). (*Vol. 33*).

Real, E., Moore, S., Selle, A., Saxena, S., Suematsu, Y. L., Tan, J., Le, Q. V., & Kurakin, A. (2017). Large-scale evolution of image classifiers. In *Proceedings of the international conference on machine learning* (pp. 2902–2911).

Selvam, K. P., Phothilimthana, P. M., Abu-El-Haija, S., Perozzi, B., & Brorsson, M. (2024). Can LLMs enhance performance prediction for deep learning models? In *Proceedings of the workshop on advancing neural network training: Computational efficiency, scalability, and resource optimization*.

Simonyan, K., & Zisserman, A. (2015). Very deep convolutional networks for large-scale image recognition. In *Proceedings of the international conference on learning representations*.

Tran, L.-T., Uddin, A. F. M. S., Jang, Y., & Bae, S.-H. (2026). KT-NAS: Knowledge transfer for efficient neural architecture search. *Applied Sciences*, *16*(2), 623.

White, C., Neiswanger, W., & Savani, Y. (2021a). BANANAS: Bayesian optimization with neural architectures for neural architecture search. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 10293–10301). (*Vol. 35*).

White, C., Nolen, S., & Savani, Y. (2021b). Exploring the loss landscape in neural architecture search. In *Proceedings of the thirty-seventh conference on uncertainty in artificial intelligence* (pp. 654–664). (*Vol. 161*).

Xiang, X., Jing, K., & Xu, J. (2024). A neural architecture predictor based on GNN-enhanced transformer. In *Proceedings of the international conference on artificial intelligence and statistics* (pp. 1729–1737).

Xie, X., Sun, Y., Liu, Y., Zhang, M., & Tan, K. C. (2023). Architecture augmentation for performance predictor via graph isomorphism. *IEEE Transactions on Cybernetics*, *54*(3), 1828–1840.

Xu, Y., Xie, L., Zhang, X., Chen, X., Shi, B., Tian, Q., & Xiong, H. (2020). Latency-aware differentiable neural architecture search. arXiv:2001.06392.

Xue, Y., & Zhu, A. (2024). An effective surrogate-assisted rank method for evolutionary neural architecture search. *Applied Soft Computing*, *167*, 112392.

Yan, S., Zheng, Y., Ao, W., Zeng, X., & Zhang, M. (2020). Does unsupervised architecture representation learning help neural architecture search? (pp. 12486–12498). (*Vol. 33*).

Yang, C., Wang, X., Lu, Y., Liu, H., Le, Q. V., Zhou, D., & Chen, X. (2024). Large language models as optimizers. In *Proceedings of the international conference on learning representations*.

Yang, Z., Wang, Y., Chen, X., Shi, B., Xu, C., Xu, C., Tian, Q., & Xu, C. (2020). CARS: Continuous evolution for efficient neural architecture search. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 1829–1838).

Ye, H., Wang, J., Cao, Z., Berto, F., Hua, C., Kim, H., Park, J., & Song, G. (2024). ReEvo: Large language models as hyper-heuristics with reflective evolution. In *Proceedings of the advances in neural information processing systems* (pp. 43571–43608). (*Vol. 37*).

Ye, P., Li, B., Li, Y., Chen, T., Fan, J., & Ouyang, W. (2022). β-DARTS: Beta-decay regularization for differentiable architecture search. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition* (pp. 10874–10883).

Yu, B., Li, Z., Cao, Y., Wu, C., Qi, J., & Wu, L. (2024). YOLO-MPAM: Efficient real-time neural networks based on multi-channel feature fusion. *Expert Systems with Applications*, *252*, 124282.

Yu, J., Gao, Y., Yang, H., Tian, Z., Zhang, P., & Zhu, X. (2025). Automated graph anomaly detection with large language models. *Knowledge-Based Systems*, *324*, 113809.

Yuan, G., Wang, B., Xue, B., & Zhang, M. (2023). Particle swarm optimization for efficiently evolving deep convolutional neural networks using an autoencoder-based encoding strategy. *IEEE Transactions on Evolutionary Computation*, *28*(5), 1190–1204.

Zhang, P., Wu, J., Yang, H., Zhou, C., & Tian, Z. (2025). Large language models enhanced graph neural architecture search for quadratic unconstrained binary optimization. *Knowledge-Based Systems*, *330*, 114520.

Zhang, R., Zhang, P.-Y., Gao, M.-R., Ma, J.-Z., & Pan, L.-H. (2024). Low-cost architecture performance evaluation strategy based on pixel difference degree contrast measurement. *Applied Soft Computing*, *155*, 111440.

Zhou, X., Wu, X., Feng, L., Lu, Z., & Tan, K. C. (2025). Design principle transfer in neural architecture search via large language models. In *Proceedings of the AAAI conference on artificial intelligence* (pp. 23000–23008). (*Vol. 39*).

Zoph, B., Vasudevan, V., Shlens, J., & Le, Q. V. (2018). Learning transferable architectures for scalable image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 8697–8710).